Homework 4: Lexical Analysis

Homework 4 Due: 2/16

For the next three homeworks, we're going to be working inside a moderately sized object oriented project. By the end of these assignments we will have

1. read a xml file, and split it up into tokens
2. assembled those tokens into a recursive XML structure
3. performed some basic operations on our XML struct.

This is a big task, so we're splitting it up into 3 parts. Todays pars it Lexical Analysis.

What is Lexical Analysis

It turns out going straight from unstructured text to a data structure we can use is actually pretty hard. Instead we break this task up into 2 parts. The first part is to split the text file up into "words". We're going to call these words *tokens*.

For our xml files, the tokens are pretty straightforward. We have 4 kinds

1. a `BEGIN_TAG` of the form `<name>`
2. an `END_TAG` of the form `</name>`
3. an identifier that is just upper or lower case letters
4. a number, which is just digits 0 through 9

Real xml files can be much more complex, but this is a good starting point for us.

I've given you a simple xml file called `text.xml`. The contents of the file are

```
<person>
 <name>Homestar</name>
 <age>26</age>
 <tetris>sure</tetris>
 <pamcakes>
   <amount>wagon_full</amount>
   <place>champeemchip</place>
   <see_try>true</see_try>
 </pamcakes>
 <soggy_bread>eww</soggy_bread>
</person>
```

When you run your `./hw4 < test.xml`, you should see the following output.

```
Tokens found:
<person> <name> [ID = Homestar] </name> <age> [NUM = 26] </age> <tetris>
[ID = sure] </tetris> <pamcakes> <amount> [ID = wagon_full] </amount>
<place> [ID = champeemchip] </place> <see_try> [ID = true] </see_try>
</pamcakes> <soggy_bread> [ID = eww] </soggy_bread> </person>
```

The begin and end tags will look like normal XML, but the numbers and identifiers have `[ ]` to distinguish them as tokens.

How do we make tokens?

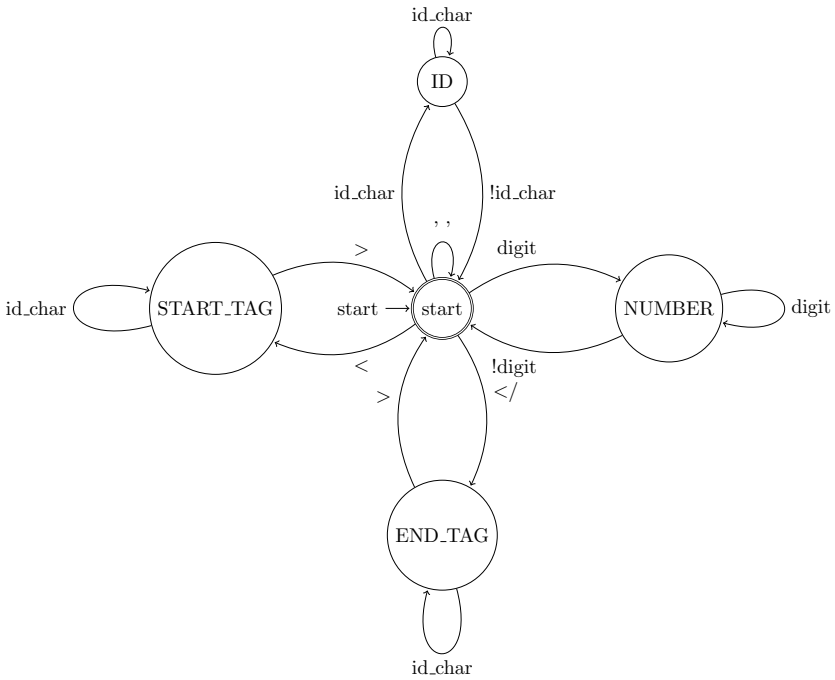
You can see the `Token` data structure in `token.h`. I have defined four different types of tokens `BEGIN_TAG, END_TAG, ID, NUM`. I have also given you a few helper functions

- `id_token(string)` creates a new ID token
- `num_token(int)` creates a new NUM token
- `start_tag_token(string)` creates a new `BEGIN_TAG` token
- `end_tag_token(string)` creates a new `END_TAG` token

You can use these functions to construct the actual token objects.

Now that I can make a token object, I need to be able to turn the file into tokens. We'll read the file from standard input (`cin`), and we have a really handy way of describing how we should split up the file with a *finite state machine*. The machine for this assignment is below. There's a lot going on here, but once you know how to read it, it's really pretty simple. We start at the circle labeled `start`. The labels on the arrows tell us where we should go if we read that character. For example, if I read a letter, then I should go to the ID state, If I read a $<$ /, then I should go to the `END_TAG` state. While I'm at the `END_TAG` state, I keep reading characters until I reach the closing $>$. Once I've read that,

I can transition back to the start state. When I transition back, I have read all of the content of the end tag, so I can make an end tag token.



There are a lot of information here, so let's see how this works with an example. Suppose I have the token `</food>`

- first we read the <, so it's either a start or end tag
- next we read the /, so it's an end tag, we transition to `END_TAG`
- we read `f, o, o, d`, which are all letters, so we stay in `END_TAG`
  - and put these letters into a string to make the token.
- finally we read a > so we transition back to the `START` state, and make a new `end_tag_token()`.

At this point it's completely reasonable to ask "the crap is a STATE? and how do I transition between them?" Fair question. It turns out a state is just a concept that we made up. We just want to separate the different pieces of our code based on the type of token we're trying to read. You can do this in a bunch of different ways.

- have a state variable that's either READING_ID, READING_NUM, READING_START, READING_END
- make a separate function for each state
- have a big if/else chain.

As long as you separate the different tasks, you should be fine. I personally like the function approach.

What goes wrong

Ok, but what if someone doesn't make a valid xml file, let `<white space in here>` Then, we need to tell the user. We're going to handle this in the traditional way. Giving up and dying. I've made a `LexException` class we can use for this. If you encounter an error, you should throw an exception with

```
throw LexException(Error message goes here);
```

Ok, but what do I actually do?

In the file `lex.cpp` you need to complete the `read_tokens` function. This should read the input stream `in` that I pass in, you should then turn that stream into a vector of tokens, and return that vector.

You just need to turn in your `lex.cpp` file. This means that you shouldn't modify any other files.

Technical notes

- a `digit` is any character between '0' and '9'
- a `letter` is any character between 'A' and 'Z' or 'a' and 'z'
- a `id_char` is any character between a `letter` or `digit`
- a `whitespace` character is any of ' ', '\t', '\n', '\r'
- the start state can just ignore any `whitespace` characters
- because the start and end tags both start with <, you need to look at the next character to decide which state you should go to.
  You might find `in.peek()` helpful here.
- You can build up a string with repeated concatenation, like java. But, if you want to do things the C++ way, you should look into `stringstream`
- Yeah, how do you make a number out of reading digits?
  C++ has a number of ways to do this.
- Remember, you might have a bad xml file where a tag starts, but is never closed. You need to make sure you can still read from `in`